SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 1- ANALYTICAL PROBLEM SOLVING

Course Code:	**CSA1206**	**Course Title:**	**Computer Architecture**
CO Assessed:	**CO2**	**Assessment:**	**ASSESSMENT TOOL1- ANALYTICAL PROBLEM SOLVING**
Weightage:	**45%**	**Total Marks:**	**25**
CO2 Statement:	Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non-restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems. (BL3)		
Student Name:	PATTEELA YASHWANTH KUMAR	**Reg.No.:**	192471055
Department:	CSE-BIOSCIENCE	**Date:**	28-07-2026

ANNEXURE A

1. A processor performs arithmetic operations on signed 8-bit integers using 2's complement representation. Consider two numbers: +45 and −23. Analyze in detail how the system performs both addition and subtraction, including binary conversion, alignment of sign bits, and intermediate steps. Determine whether overflow occurs, and explain how the processor detects overflow and interprets the final result.

2. A high-performance computing system replaces a ripple carry adder with a 4-bit carry lookahead adder (CLA) to reduce computation delay. Given two binary inputs, analyze how generate and propagate signals are formed and how carries are computed in advance. Compare the time delay of CLA with ripple carry addition and justify, with reasoning, why CLA improves speed in modern processors.

3. In a signed multiplication operation, a processor uses Booth's algorithm to multiply two numbers, −6 and +5. Analyze the algorithm step-by-step, including the role of the accumulator, multiplier, and Booth bit, along with arithmetic shifts. Explain how the algorithm efficiently handles signed numbers and reduces the number of addition/subtraction operations compared to traditional multiplication.

4. A processor is required to perform division of two binary numbers (e.g., 13 ÷ 3) using both restoring and non-restoring division algorithms. Analyze both methods step-by-step, showing intermediate remainders and quotient formation. Compare the two approaches in terms of number of steps, complexity, and efficiency, and explain why non-restoring division is often preferred in modern systems.

5. A scientific application requires precise handling of real numbers using the IEEE 754 standard for floating-point representation. Given a decimal number (e.g., −13.25), analyze how it is represented in both single precision and double precision formats, including sign, exponent, and mantissa fields. Further, explain how floating-point addition is performed between two such numbers, highlighting issues like normalization, rounding, and precision loss.

Code of Conduct:

I Pattteela Yashwanth kumar /192471055 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Patteela Yashwanth kumar

MARKS ALLOCATION

	Problem Understanding (20%)	**Method Selection (20%)**	**Solution Process (25%)**	**Accuracy of Calculation (20%)**	**Interpretation of Results (15%)**
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	**Weightage**	**Level 4 – Exemplary**	**Level 3 – Proficient**	**Level 2 – Developing**	**Level 1 – Beginning**
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

1.Addition and Subtraction using 2's Complement (Signed 8-bit Integers)

Given

- A = +45
- B = −23

(a) Binary Conversion

+45

Decimal = 45

Binary (8-bit)

00101101

−23

23 in binary

00010111

Take 2's complement

1's complement =11101000

+1

11101001

Therefore,

−23 =11101001

Addition

+45 =00101101

−23 =11101001

00010110

Carry discarded. Result

00010110 = +22

Decimal Verification

45 + (−23)=22

Subtraction

Compute

45−(−23)=45+23

23 in binary

00010111

Addition

00101101

00010111

01000100

Result

01000100 =68

Overflow Detection

Overflow occurs only when

- Adding two positive numbers gives negative
- Adding two negative numbers gives positive

Here Positive +

Negative No

overflow.

Similarly

Positive + Positive = Positive

Result =68

Still within

−128 to +127 No

overflow.

Final Results

Operation	Binary Result	Decimal	Overflow
45+(-23)	00010110	22	No
45−(-23)	01000100	68	No

2. Carry Look Ahead Adder (CLA)

Given

4-bit inputs

A=1011

B=0110

Carry input

C0=0

Step 1 Generate (G)

Gi=Ai·Bi

Bit	A	B	Gi
0	1	0	0
1	1	1	1
2	0	1	0
3	1	0	0

Step 2 Propagate (P)

Pi=Ai⊕Bi

Bit	Pi
0	1
1	0
2	1
3	1

Step 3 Carry Equations

C1=G0+P0C0

=0+(1×0)=0

C2=G1+P1G0+P1P0C0

=1+0+0=1

C3=G2+P2G1+P2P1G0+P2P1P0C0

=0+1×1=1

C4=G3+P3G2+P3P2G1+...

=1

Sum

Si=Pi⊕Ci

Result

10001

Decimal

11+6=17
Correct.

Delay Comparison

Ripple Carry Adder:

Carry travels through every full adder.
Delay
O(n)

Carry Look Ahead Adder

Carries computed simultaneously.
Delay
O(log n)

Advantages of CLA

- Faster computation
- Parallel carry generation
- Used in modern CPUs
- Reduces propagation delay.

3. Booth's Multiplication (−6 × +5)

Given

Multiplicand M = −6
Multiplier Q = +5
Use **4-bit representation**
M =1010
−M=0110
Q =0101
A =0000
Q-1=0

Booth Rules

Q0Q−1	Operation
00	No operation
11	No operation
01	A=A+M
10	A=A−M

Initial

A=0000
Q=0101
Q-1=0

Cycle 1
Q0Q−1=10 A=A−M
0000−1010
=0110
Arithmetic Shift
A=0011
Q=0010
Q−1=1
Cycle 2
Q0Q−1=01
A=A+M
Shift
A=1110
Q=1001
Cycle 3
10
Subtract
Shift
Cycle 4 Repeat
Final product
11100010
Decimal
11100010 = −30
Verification
−6×5=−30
Correct.

Booth Algorithm Advantages

- Handles signed numbers automatically.
- Reduces number of additions/subtractions.
- Efficient for consecutive 1s.
- Faster than ordinary shift-and-add multiplication.

4. Restoring and Non-Restoring Division (13 ÷ 3)

Dividend

13=1101

Divisor

3=0011

(a) Restoring Division

Initial

A=0000

Q=1101

M=0011

Step 1

Shift left

Subtract divisor

If

A<0

Restore

A=A+M

Quotient bit

0

Continue

Repeat 4 cycles.

Final

Quotient=0100

Remainder=0001

Decimal

13÷3=4 remainder1

(b) Non-Restoring Division

Initial

A=0000

Q=1101

If

A≥0

Subtract divisor.

If

A<0

Next cycle add divisor.

No restoring step after every subtraction.

After four cycles

Q=0100

R=0001

Comparison

Feature	Restoring	Non-Restoring
Restore operation	Yes	No
Speed	Lower	Higher
Complexity	Simple	Moderate
Operations	More	Less
Efficiency	Lower	Higher

Why Non-Restoring is Preferred

- Fewer arithmetic operations.
- Reduced execution time.
- Better hardware efficiency.
- Widely used in modern processors.

5. IEEE 754 Floating Point Representation (−13.25)

Decimal Number

−13.25

Binary Conversion

Integer

13=1101

Fraction

0.25=0.01

Therefore

1101.01_2

Normalize

1.10101×2^3

(a) Single Precision (32-bit)

Sign

Negative

S=1

Exponent

Bias

127

Exponent

3+127=130

Binary

10000010

Mantissa

Remove leading 1
10101000000000000000000

Final IEEE-754
1
10000010
10101000000000000000000

(b) Double Precision (64-bit)

Bias
1023
Exponent
1026
Binary
10000000010
Mantissa
1010100000000000000000000000000000000000000000000000
Final
1
10000000010
1010100000000000000000000000000000000000000000000000

Floating Point Addition Steps

Suppose
13.25+5.5 **Steps:**

1. Convert both numbers into IEEE-754 format.
2. Compare exponents.
3. Align exponents by shifting the smaller mantissa.
4. Add mantissas.
5. Normalize the result.
6. Round according to IEEE-754 (typically **round to nearest, ties to even**).
7. Store sign, exponent, and mantissa.

Issues in Floating-Point Arithmetic

1. Normalization

Ensures the number is stored in the standard normalized form:
1.xxxxx × 2^e

2. Rounding

When mantissa exceeds available bits, IEEE-754 rounds the value, introducing a small approximation error.

3. Precision Loss

Occurs when:

> Very large and very small numbers are significant digits are discarded during alignment.

Example:

$1000000.0 + 0.0001 \approx 1000000.0$

The smaller value may be lost due to limited precision.